INFOSYS SE ASSESSMENT 2026 • OFFICIAL PREPARATION VAULT

📊 Top 100 Data Structures & Algorithms Sheet for Infosys DSE & SP

Section: 07_[OPTIONAL_DSE_SP_ONLY]_Coding_Prep | **Drive Date:** 5th & 6th November 2026 | **Status:** 100% Solved & Verified

📊 Top 100 Data Structures & Algorithms Sheet for Infosys DSE & SP

> **Curated by Amogh Sir & CampusMonk Master Faculty**
> **Target:** 100% Pass Guarantee for Infosys Digital Specialist Engineer & Specialist Programmer Coding Assessments
> **Languages:** Complete Python, Java, and C++ Implementation Blueprints

---

1. Top 25 Arrays & String Problems (Infosys High-Frequency)

Problem 01: Next Permutation

- **Infosys Real Exam Frequency:** Extremely High (HackWithInfy & DSE)
- **Problem Statement:** Rearrange numbers into the lexicographically next greater permutation. If no such arrangement is possible, rearrange into the lowest possible order (sorted in ascending order).
- **Time Complexity:** $O(N)$, **Space Complexity:** $O(1)$.
- **Optimal Algorithm:**
  1. Find the first decreasing element from the right ($A[i] < A[i+1]$).
  2. Find the element just greater than $A[i]$ to its right ($A[j] > A[i]$).
  3. Swap $A[i]$ and $A[j]$.
  4. Reverse the subarray from $i+1$ to end.

```
def next_permutation(nums: list[int]) -> None:
    i = len(nums) - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    if i >= 0:
        j = len(nums) - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i]
    nums[i + 1:] = reversed(nums[i + 1:])
```

Problem 02: Maximum Subarray Sum (Kadane's Algorithm with Subarray Retrieval)

- **Problem Statement:** Find the contiguous subarray (containing at least one number) which has the largest sum and return its sum along with starting/ending indices.
- **Time Complexity:** $O(N)$, **Space Complexity:** $O(1)$.

```
def max_sub_array(nums: list[int]) -> tuple[int, int, int]:
    max_so_far = nums[0]
    curr_max = nums[0]
    start = end = s = 0

    for i in range(1, len(nums)):
        if nums[i] > curr_max + nums[i]:
            curr_max = nums[i]
            s = i
        else:
            curr_max += nums[i]

        if curr_max > max_so_far:
            max_so_far = curr_max
            start = s
            end = i

    return max_so_far, start, end
```

Problem 03: Trapping Rain Water

- **Infosys Frequency:** Very High (Appeared in Oct 2025 Slot 2)
- **Two-Pointer Approach:** $O(N)$ time, $O(1)$ space.

```
def trap(height: list[int]) -> int:
    if not height:
        return 0
    left, right = 0, len(height) - 1
    left_max, right_max = height[left], height[right]
    water = 0

    while left < right:
        if left_max < right_max:
            left += 1
            left_max = max(left_max, height[left])
            water += left_max - height[left]
        else:
            right -= 1
            right_max = max(right_max, height[right])
            water += right_max - height[right]

    return water
```

---

2. Top 25 Dynamic Programming & Greedy Problems

Problem 04: 0/1 Knapsack Problem

```
def knap_sack(W: int, wt: list[int], val: list[int], n: int) -> int:
    dp = [0] * (W + 1)
    for i in range(n):
        for w in range(W, wt[i] - 1, -1):
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i])
    return dp[W]
```

Problem 05: Longest Common Subsequence (LCS)

```
def lcs(s1: str, s2: str) -> int:
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```